LAB 4 Theory

Adesijibomi Aderinto

2025-06-17

Task 1 - Gridworld

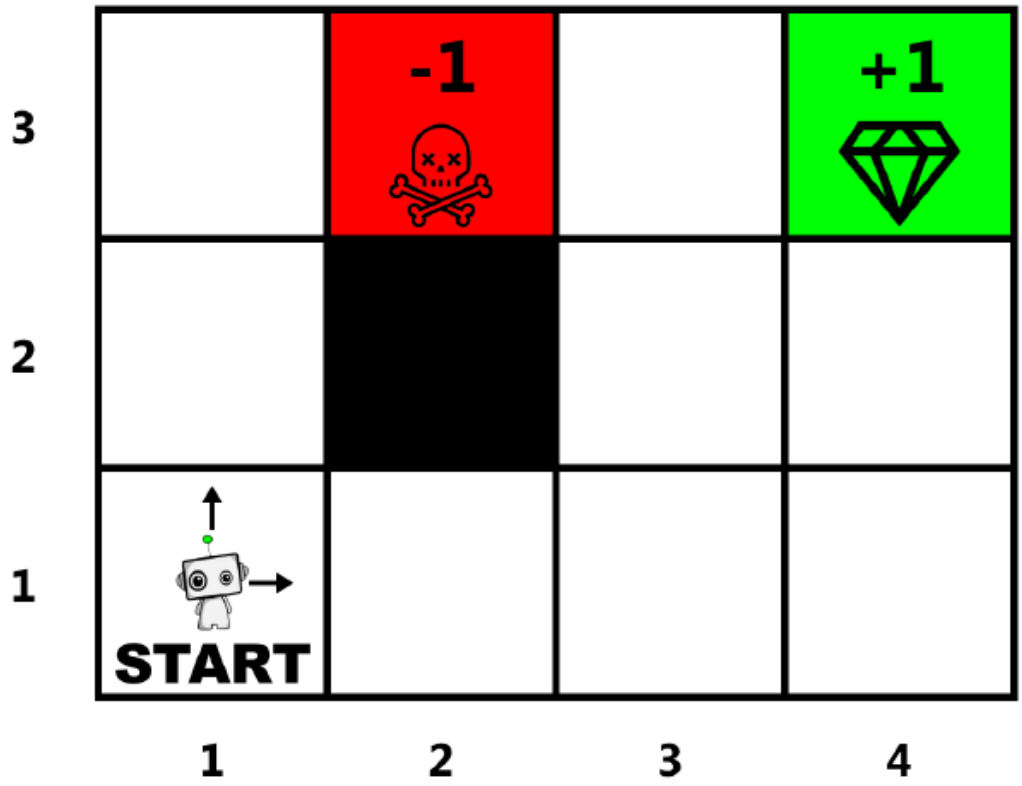

Figure 2: Gridworld

Figure 1: Gridworl

$$V(s) = \max_{\pi} \mathbb{E}\left[\sum_{t=0}^{H} \gamma^t R(S_t, A_t, S_{t+1}) \mid \pi, s_0 = s\right]$$

Task 1.2

Assume that the transition function is equal to 1, $\gamma = 0.9$ and H = 100.

- **v(4,3)** $= V(4,3) = R(4,3) = +1$
- **v(2,3)** $= V(2,3) = R(2,3) = -1$
- **v(3,3)** $= V(3,3) = \gamma \cdot V(4,3) = 0.9 \cdot 1 = 0.9$
- **v(3,2)** $= V(3,2) = \gamma \cdot V(3,3) = 0.9 \cdot 0.9 = 0.81 \longrightarrow V(3,1) = \gamma \cdot V(3,2) = 0.9 \cdot 0.81 = 0.729$
- **v(1,1)** $=$ (1,1)→(2,1)→(3,1)→(3,2)→(3,3)→(4,3) $V(2,1) = \gamma \cdot V(3,1) = 0.9 \cdot 0.729 = 0.6561$

$$V(1,1) = \gamma \cdot V(2,1) = 0.9 \cdot 0.6561 = 0.59049$$

Task 1.3

Assume that the transition function is equal to 0.8, $\gamma = 0.7$ and H = 100.

Task 2

- **Exploration** means trying out new actions or strategies to discover more information about the environment, for example, trying an action you haven't tried before to see if it might yield a better reward.
- **Exploitation** means using the current knowledge you have to pick the best-known action that maximizes your reward based on what you've learned so far.
- **Credit-Assignment** problem is about figuring out which past actions caused future rewards in environments where rewards are delayed.
- **Monte Carlo (MC) methods** wait until the end of an episode to assign credit. They calculate the total return (sum of rewards) after the episode finishes and then assign that return to all state-action pairs encountered during the episode. It can also means that the current state contains all the information needed to predict the next state, without needing the full history, i.e future is independent of the past given the present state.
- **Chess environment** satisfies the Markov property because the agent's next state and received reward depend solely on its current position and chosen action, without requiring any information about prior states or actions. The presence of goal, penalty, and blocked states influences the immediate reward and allowed movements, but these effects are fully captured by the current state.
- **Q-learning** is a classic reinforcement learning algorithm that learns a table (Q-table) mapping each state-action pair to an expected future reward. It works well for environments with small, discrete state and action spaces because it explicitly stores and updates values for every possible state-action pair.
- **Deep Q-learning** extends Q-learning by using a deep neural network to approximate the Q-function instead of a table. This allows it to handle large or continuous state spaces (like images or high-dimensional data) where a Q-table would be impossible to maintain. The neural network generalizes learning across states, making Deep Q-learning suitable for complex tasks like playing video games or robotics